# Voice Agent Submission Report

## 1. Project Summary

This project implements a voice-driven rider onboarding agent that calls delivery-rider leads, qualifies them against an approved checklist, answers pay questions from an approved FAQ, books onboarding slots, and reports outcomes back to a CRM and call-status webhook system.

The solution combines:

• a Python FastAPI backend for deterministic business logic

• a SQLite datastore for leads, slots, bookings, and CRM state

• a Sarvam-based voice agent for outbound conversational flow

• a scheduler for working-hours dialing and retry logic

• a Streamlit dashboard for operational visibility

## 2. Verified Runtime Results

The backend was executed and validated directly on the local host.

### *Local health check*

Observed result:

• URL: http://127.0.0.1:8000/health

• HTTP status: 200 OK

• Response: {"status":"ok","version":"0.1.0"}

This confirms the application is running successfully on port 8000.

### *Route behavior validation*

Additional checks confirmed expected route protection and method behavior:

• /slots?city=delhi -> 401 Unauthorized

• /bookings -> 405 Method Not Allowed

These responses are correct because the backend enforces an API key on protected routes and only allows POST for booking creation.

### *Public tunnel validation*

A quick Cloudflare tunnel was created and validated:

• Public URL: https://passage-fees-hiking-applicants.trycloudflare.com

• Health path: https://passage-fees-hiking-applicants.trycloudflare.com/health

• Observed HTTP status: 200 OK

• Response: {"status":"ok","version":"0.1.0"}

This shows the tunnel is live and correctly forwarding requests to the app.

## 3. Functional Implementation

### *FastAPI backend*

The backend is defined in src/rider_agent/api/app.py and exposes the following routes:

• GET /health

• GET /slots

• POST /bookings

• POST /crm/outcomes

• POST /webhooks/call-status

The backend uses an API-key requirement for protected routes and performs compliance-driven validation before processing requests.

### *CLI operations*

The CLI supports the following operational commands:

• rider-agent serve

• rider-agent add-lead

• rider-agent import-leads

• rider-agent add-dnd

• rider-agent seed-slots

• rider-agent run-scheduler

• rider-agent dial-lead

• rider-agent stats

This enables demo execution, lead import, scheduler control, and operational management from the terminal.

### *Scheduling and retry logic*

The system is designed to:

• restrict outbound dials to working hours

• retry unanswered or failed calls with backoff

• skip DND numbers

• maintain deterministic lead state and booking integrity

• avoid duplicate or conflicting CRM outcomes

### *Qualification and booking logic*

The agent qualifies each lead using a fixed checklist and the approved rider FAQ. Booking is only allowed under guardrails such as:

• valid city match

• open slot availability

• lead qualification status

• no duplicate bookings for the same interaction

## 4. Architecture Summary

The application follows a clear layered structure:

1. Lead sources feed lead data into the database.

2. The scheduler decides which leads to call and when.

3. The Sarvam voice agent speaks to the lead and collects qualification answers.

4. The backend validates and executes slot booking and CRM updates.

5. The dashboard exposes aggregate operational state.

## 5. Setup Instructions

### *Prerequisites*

• Python 3.11+

• pip

• terminal access

• working Sarvam workspace credentials

### *Install dependencies*

python -m venv .venv

source .venv/bin/activate

pip install -U pip

pip install -e ".[dev,dashboard]"

### *Start the backend*

rider-agent serve

This starts the API on:

• http://127.0.0.1:8000

### *Quick tunnel for testing*

cloudflared tunnel --url http://127.0.0.1:8000

Then use the generated hostname from Cloudflare in PUBLIC_BASE_URL.

### *Example public route checks*

https://.trycloudflare.com/health

https://.trycloudflare.com/slots?city=delhi

https://.trycloudflare.com/bookings

## 6. Example Commands

### *Add a lead*

rider-agent add-lead "Asha" 9876543210 Bengaluru --language Hindi

### *Dial immediately*

rider-agent dial-lead 9876543210

### *Run scheduler loop*

rider-agent run-scheduler --loop

### *Open dashboard*

streamlit run dashboard/app.py

## 7. Sample Snapshots

### *Snapshot A: Local health endpoint response*

{"status":"ok","version":"0.1.0"}

### *Snapshot B: Public tunnel health response*

{"status":"ok","version":"0.1.0"}

### *Snapshot C: Protected route guardrail*

{"detail":"Invalid API key."}

### *Snapshot D: Method restriction*

{"detail":"Method Not Allowed"}

## 8. Submission Conclusion

The project is functionally working as a voice-led rider onboarding system. The local backend is healthy, the public tunnel is live, and the main API routes are active and protected by proper guardrails. The project is suitable for a demo or submission where the goal is to show an implemented, working AI-assisted onboarding workflow.

## 9. Important Note

The generated quick-tunnel URL is temporary and may change if the tunnel is restarted. For a stable production hostname, a named Cloudflare tunnel is recommended instead of a temporary quick tunnel.